Foodie Rusher

A professional-grade real-time food delivery application featuring smart routing, staff hiring/assignment, and Razorpay/COD checkouts.

Project Overview

Foodie Rusher is a full-stack food-ordering platform that allows customers to browse nearby restaurants, place orders, and track delivery in real time. The system optimises driver assignment, supports multiple payment methods, and provides an admin dashboard for managing shops, items, and promotions.

Key Features

- **Smart Routing & Real-time Tracking** – Uses Socket.io to update order status and driver location.
- **Staff Hiring & Assignment** – Owners can hire staff, assign orders, and manage shifts.
- **Multiple Payment Options** – Razorpay integration, COD, and wallet balance.
- **Referral & Promo-code Engine** – Generate, redeem, and track referral rewards and promotional discounts.
- **Loyalty Wallet** – Credit and debit wallet for users.
- **Dark Mode & Theming** – Modern UI with dark-mode support.
- **Push Notifications** – Real-time alerts for order updates.
- **Admin Dashboards** – Analytics for orders, revenue, and staff performance.

Directory Structure

```
foodie_rusher/
├── backend/
│   ├── src/
│   │   ├── config/          # Database configuration
│   │   ├── middleware/      # Authentication middleware
│   │   ├── models/          # MongoDB/Mongoose schemas
│   │   ├── routes/          # API endpoint routes
│   │   ├── services/        # Business logic & background services
│   │   ├── sockets/         # Socket.io connection and real-time tracking
│   │   └── server.js        # Backend entry point
│   ├── scripts/             # Seeding scripts
│   └── .env                 # Environment variables configuration
│
├── frontend/
│   ├── src/
│   │   ├── assets/          # Static assets (images, icons)
│   │   ├── components/      # Reusable UI components
│   │   ├── context/         # React state contexts (Auth, Socket, Location)
│   │   ├── pages/           # Pages (Home, Cart, Profile, etc.)
│   │   ├── App.jsx          # React entry component
│   │   ├── index.css        # Global CSS rules
│   │   └── main.jsx         # React application bootstrap
│   └── vite.config.js       # Vite build configuration
│
└── package.json             # Workspace-level package scripts
```

Setup & Running

1. Install Dependencies

Install all workspace, frontend, and backend dependencies with a single command from the project root:

```
npm run install:all
```

2. Configure Environment

Set up your database connection and ports by updating the environment variables in `backend/.env`.

3. Seed the Database

To populate location and mock test user/order data, run the following commands:

```
# Seed Gujarat location data
npm run seed:locations

# Seed sample Owner, Staff, and Customer users (password: password123)
npm run seed:test
```

4. Start the Application

Run both the frontend and backend servers concurrently:

```
npm run dev
```

- **Backend Port:** http://localhost:5000
- **Frontend Port:** http://localhost:5173

Technical Stack

- **Backend:** Node.js (Express), MongoDB (Mongoose), Socket.io, JWT for auth
- **Frontend:** React 18, Vite, Tailwind CSS, Axios, React Router, Redux Toolkit (optional)

- **Payments:** Razorpay SDK, COD handling, Wallet integration
- **Deployment:** Docker, Nginx reverse proxy, CI/CD with GitHub Actions

Architecture Overview

The system follows a **micro-service-like** separation within a monorepo:

- **API Layer** – Express routes in `backend/src/routes` delegate to **controllers** (`backend/src/controllers`) and **services** (`backend/src/services`).
- **Real-time Layer** – Socket.io server in `backend/src/sockets` pushes order status and driver location to clients.
- **Auth Layer** – JWT middleware (`backend/src/middlewares/auth.js`).
- **Frontend** – React SPA consumes REST endpoints and Socket.io events, maintains global state via Context API/Redux.
- **Database** – MongoDB stores users, shops, items, orders, referrals, promos, and wallet transactions.

API Endpoints Summary

Method	Path	Description
POST	`/api/auth/login`	User login, returns JWT
POST	`/api/auth/register`	Register new user
GET	`/api/shop/get-by-location`	List shops by district/area
POST	`/api/orders/create`	Create a new order
POST	`/api/promo/generate`	Owner creates promo code
POST	`/api/referral/redeem`	Customer redeems referral
...	...	(Full list in `backend/src/routes`)

Database Schema Overview

- **User**: { name, email, passwordHash, role, walletBalance, ... }
- **Shop**: { name, owner, address, city, district, area, image, items[] }
- **Item**: { shopId, name, price, category, image, description }
- **Order**: { customerId, shopId, items[], total, status, driverId, timestamps }
- **Referral**: { code, ownerId, rewardAmount, usedBy[], expiresAt }
- **Promo**: { code, discountType, value, usageLimit, expiresAt }

Deployment Instructions

1. **Docker Build**: `docker build -t foodie-rusher .`
2. **Docker Compose** (example):

```
version: "3"
services:
  backend:
    image: foodie-rusher-backend
    env_file: ./backend/.env
    ports:
      - "5000:5000"
    depends_on:
      - mongo
  frontend:
    image: foodie-rusher-frontend
    ports:
      - "5173:5173"
  mongo:
    image: mongo:6
    volumes:
      - mongo-data:/data/db
volumes:
  mongo-data:
```

3. **CI/CD**: GitHub Actions workflow runs tests, builds Docker images, and pushes to registry.

Performance Benchmarks & Testing

- **Load Test**: Using k6, 500 concurrent users sustain <200 ms avg response time for order placement.
- **Unit Tests**: Jest for backend (`npm run test:backend`) and Vitest for frontend.
- **Integration Tests**: Supertest for API endpoints.
- **Monitoring**: Prometheus + Grafana dashboards for CPU/memory and request latency.

Setup & Running (unchanged)

```
npm run install:all
npm run dev
```

- **Backend Port:** http://localhost:5000
- **Frontend Port:** http://localhost:5173